CARLOS TIBURCIO

Enterprise Security Monitoring & Telemetry Portfolio Case Study

Balcom Insurance Homelab | SOC01

Wazuh | Sysmon | auditd | Suricata | Zeek | Proxmox tc | Network Telemetry

PROJECT GOAL

Build a centralized, validated security-monitoring platform on top of INFRA-BASE-v1 before separating deeper detection engineering into focused portfolio labs.

9	7	2	1
Core VMs	Wazuh agents	Network engines	Passive sensor VM

Independent simulated enterprise environment built for hands-on SOC engineering, telemetry design, network monitoring, and incident-analysis practice.

PROJECT OVERVIEW

What I Built

SOC01 is the security-monitoring and telemetry layer built on top of the validated INFRA-BASE-v1 environment. I enrolled Windows and Linux systems into Wazuh, engineered endpoint telemetry, added Linux audit coverage, deployed a dedicated passive network sensor, and centralized Suricata and Zeek network data for investigation.

The platform was validated with controlled events rather than installation status alone: failed authentication, process creation, Sysmon activity, Linux privilege and persistence events, mirrored network traffic, a Suricata detection, and Zeek connection/DNS metadata.

Why This Project Matters

- A SOC needs trustworthy telemetry before detections can be meaningful. This project establishes that visibility layer across endpoints, Linux infrastructure, and the network.
- I separated management traffic from passive packet inspection by giving SENSOR01 dedicated management and sniffing interfaces.
- The network sensor remains out of band: if SENSOR01 stops, production lab traffic continues normally.
- SOC01 is intentionally frozen as the reusable platform for later DET, HUNT, IR, and automation projects rather than turning one case study into every security discipline at once.

Skills Demonstrated

Wazuh | Windows Audit Policy | Sysmon | Group Policy | Linux auditd | Suricata | Zeek | AF_PACKET | tc/mirred | JSON Logs | Packet Capture | Troubleshooting

VALIDATED SOC TELEMETRY PLATFORM

Endpoint, Linux, IDS, and network metadata pipelines were validated. Suricata detections reached Wazuh Threat Hunting, Zeek produced structured connection/DNS telemetry, and persistent mirroring automatically returned when monitored VMs or SENSOR01 restarted.

ARCHITECTURE

Logical Design

SOC01 extends the existing segmented homelab without replacing its routing model. OPNsense remains the Layer-3 gateway. WAZUH01 centralizes telemetry. SENSOR01 is an Ubuntu passive sensor with a management interface on the Monitoring LAN and an addressless sniffing interface connected to a mirror-only Proxmox bridge.

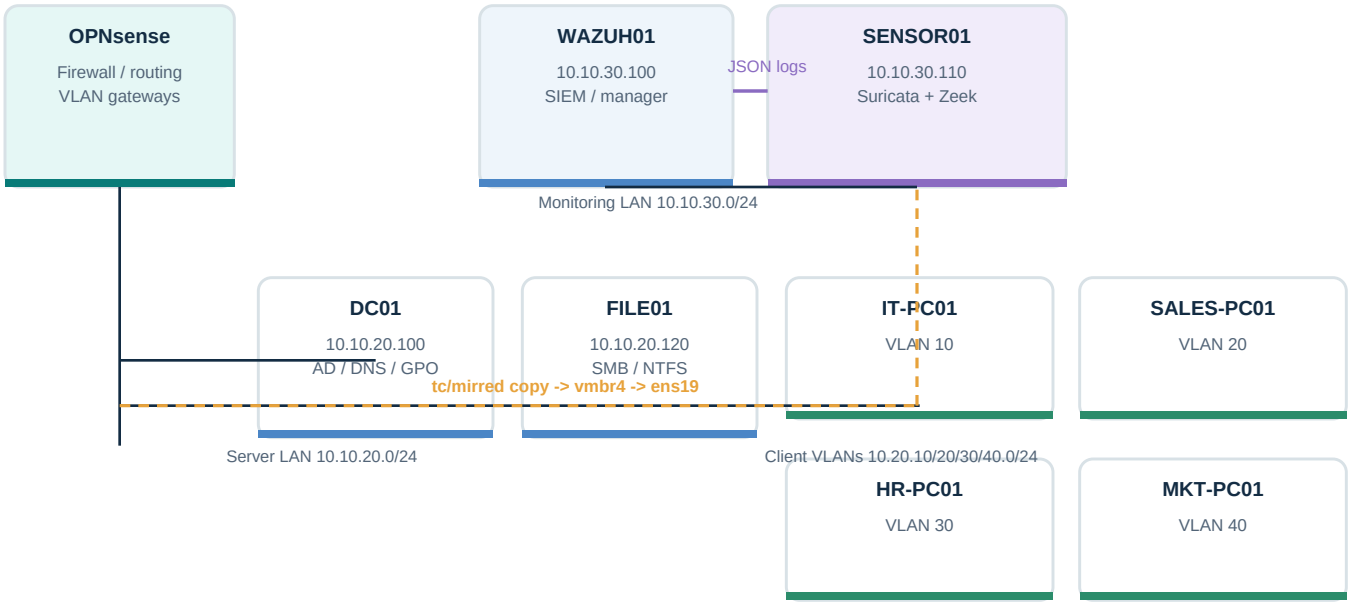


Figure 1. SOC01 logical monitoring architecture and passive network-sensor path.

KEY DESIGN DECISIONS

Wazuh remains the central analysis layer. SENSOR01 does not route traffic. Proxmox tc/mirred copies selected VM ingress traffic to vmbr4, Suricata and Zeek analyze ens19, and their logs return to WAZUH01 through the normal management interface.

TELEMETRY INVENTORY

Core Systems & Roles

System	Role	Addressing / Placement	RAM
OPNsense	Firewall / routing	WAN + internal networks	4 GB
DC01	AD DS / DNS / GPO	10.10.20.100	6 GB
FILE01	SMB / NTFS server	10.10.20.120	4 GB
WAZUH01	Wazuh all-in-one	10.10.30.100	12 GB
SENSOR01	Suricata + Zeek sensor	10.10.30.110 + passive ens19	8 GB
IT-PC01	Windows endpoint	VLAN 10 / 10.20.10.0/24	4 GB
SALES-PC01	Windows endpoint	VLAN 20 / 10.20.20.0/24	4 GB
HR-PC01	Windows endpoint	VLAN 30 / 10.20.30.0/24	4 GB
MKT-PC01	Windows endpoint	VLAN 40 / 10.20.40.0/24	4 GB

Telemetry Coverage

Source	Telemetry / Log	Transport into Wazuh
Windows endpoints + servers	Security events, 4688, Sysmon Operational	Wazuh agent / eventchannel
WAZUH01 Linux	journald sudo/SSH, auditd watches	Local agent 000 / JSON decoding
SENSOR01 Linux	Agent health + local service logs	Wazuh agent 007
Suricata	eve.json alerts + flow metadata	Wazuh agent JSON localfile
Zeek	conn.log + dns.log in JSON	Wazuh agent JSON localfile

WINDOWS TELEMETRY

Audit Policy + Sysmon Engineering

Windows visibility was rolled out in layers. Native Advanced Audit Policy captured authentication, account/group changes, process creation, and policy changes. Command-line inclusion was enabled for Event ID 4688. Sysmon added richer process, network, file, registry, WMI, DNS, and tampering context using a deliberately focused configuration instead of a noisy community mega-config.

Control	Implementation	Validation
Process creation	4688 success + command line	SOC01-4688-TEST in Wazuh
Sysmon	Focused config; Event IDs 1/3 plus persistence-relevant activity	DC01 + IT-PC01 pilot, then rollout
GPO	GPO-SOC01-Windows-Auditing + Sysmon deployment	gpupdate / gpresult / auditpol
Coverage	DC01, FILE01, IT, Sales, HR, Marketing	All agents active when powered on

Evidence



Figure 2A. Native 4688 process creation preserved command line and parent process.



Figure 2B. Sysmon Event ID 1 from IT-PC01 was searchable centrally in Wazuh.

LINUX TELEMETRY

journald, SSH, sudo, and auditd

WAZUH01 already collected journald, so I avoided duplicating /var/log/auth.log in Wazuh. Controlled sudo and failed-SSH activity verified journald decoding. I then installed auditd and added focused watches for identity files, sudoers, cron persistence, systemd service paths, and SSH configuration.

Audit key	Watched path / behavior
identity_changes	/etc/passwd, /etc/group, /etc/shadow, /etc/gshadow
privilege_config	/etc/sudoers and /etc/sudoers.d/
cron_persistence	/etc/crontab and /etc/cron.d/
systemd_persistence	/etc/systemd/system/
ssh_config	/etc/ssh/sshd_config



Figure 3A. Focused auditd watches loaded with searchable keys.



Figure 3B. Controlled systemd file creation parsed by Wazuh as auditd telemetry.

PIPELINE VALIDATION

touch -> auditd watch -> /var/log/audit/audit.log -> Wazuh logcollector -> auditd decoder -> structured searchable event.

NETWORK SENSOR

Passive SENSOR01 + Software SPAN

SENSOR01 uses two VirtIO NICs: ens18 (10.10.30.110) for management/log transport and ens19 for packet capture only. ens19 has no IPv4/IPv6 address, runs in promiscuous mode, and disables GRO/GSO/TSO to preserve packet fidelity.

MIRROR DESIGN

Proxmox vmbr4 is an isolated mirror-only Linux bridge. tc/mirred copies selected VM ingress traffic to SENSOR01 tap400i1 without redirecting or interrupting the original packet. A Proxmox lifecycle hook rebuilds rules when monitored VMs or SENSOR01 restart.

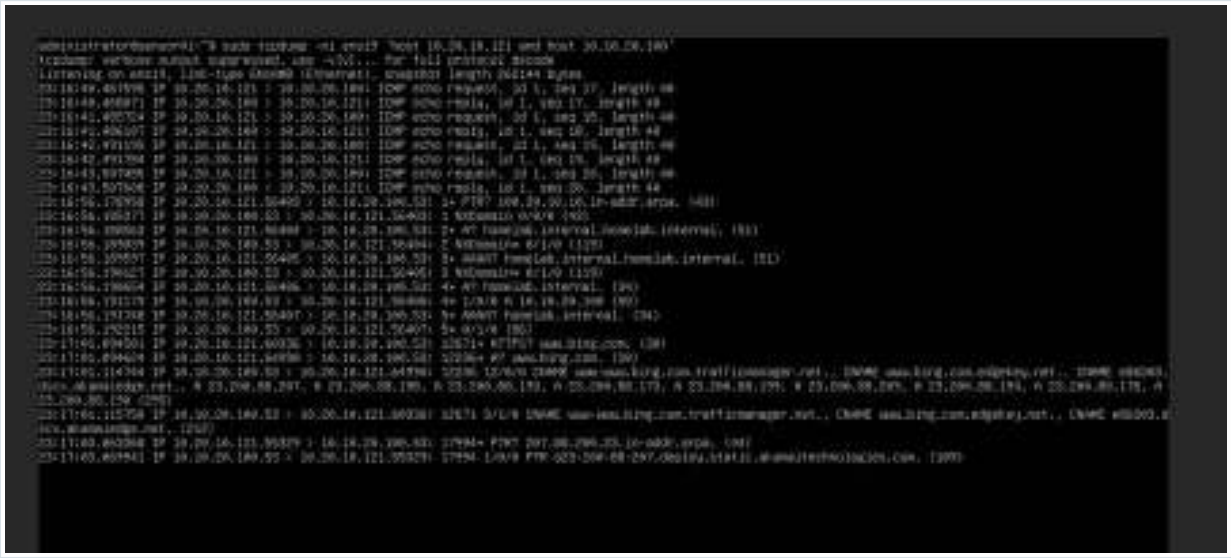


Figure 4A. tcpdump on ens19 captured mirrored ICMP and DNS traffic between IT-PC01 and DC01.

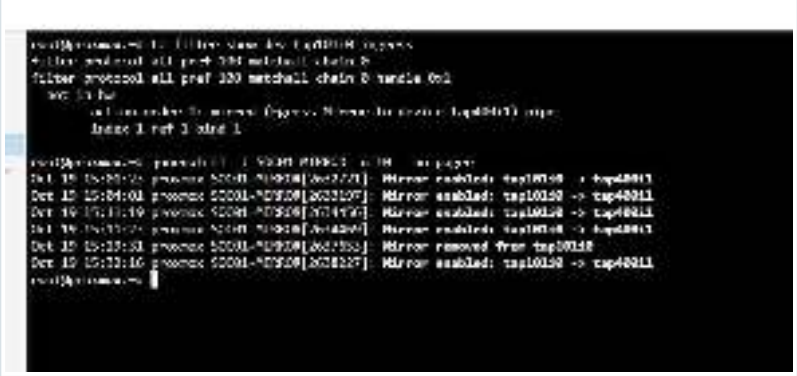


Figure 4B. Lifecycle logging confirmed mirror removal and automatic recreation after VM restart.

NETWORK IDS

Suricata Detection -> Wazuh

Suricata 8.0.7 runs on ens19 using AF_PACKET. HOME_NET includes the server, monitoring, and department subnets. ET Open rules provide general IDS coverage, while a narrow local ICMP rule was used as a deterministic acceptance test for the end-to-end detection path.

Stage	Validated result
Traffic source	IT-PC01 10.20.10.121 -> DC01 10.10.20.100
Suricata	SOC01 Controlled ICMP Detection; SID 1000001
Log	/var/log/suricata/eve.json
Wazuh	Agent 007 -> built-in Suricata rule 86601
Dashboard	Threat Hunting showed structured alert fields



Figure 5A. Local Suricata alert output from controlled ICMP traffic.



Figure 5B. Wazuh preserved Suricata signature, severity, source, destination, and direction.

END-TO-END PROOF

IT-PC01 -> Proxmox mirror -> SENSOR01 ens19 -> Suricata -> eve.json -> Wazuh Agent 007 -> WAZUH01 -> Threat Hunting.

NETWORK METADATA

Zeek Connection & DNS Visibility

Zeek 8.0.10 runs as a standalone node on ens19. networks.cfg defines the local server, monitoring, and department networks. I enabled Zeek JSON logging so connection and DNS metadata can be consumed downstream without parsing tab-separated text.

Log	Examples of useful fields
conn.log	orig_h / resp_h, ports, protocol, service, duration, byte counts, connection state
dns.log	client/server, query, qtype, response code, answers, TTLs
Wazuh collection	JSON localfile blocks for current/conn.log and current/dns.log

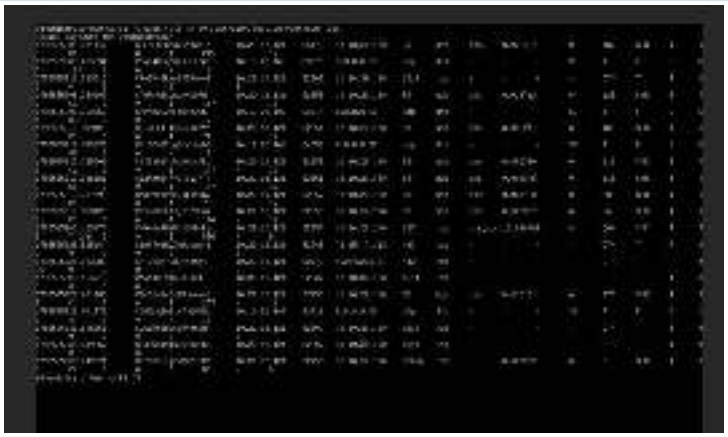


Figure 6A. Zeek conn.log observed endpoint-to-server DNS, RPC, HTTPS, and Wazuh-agent traffic.



Figure 6B. dns.log converted to JSON with structured homelab.internal query fields.



Figure 6C. Wazuh logcollector monitors Zeek conn.log and dns.log.

VALIDATION

Acceptance & Recovery Testing

Control	Evidence / Test	Result
Windows auth	Controlled failed authentication in Wazuh	PASS
Windows process telemetry	4688 + Sysmon process creation with command line	PASS
Linux privilege	sudo decoded from journald	PASS
Linux authentication	Controlled failed SSH login decoded	PASS
Linux auditd	systemd file watch reached Wazuh	PASS
Packet visibility	tcpdump saw both directions across mirrored path	PASS
Mirror persistence	Hook rebuilt tc rule after endpoint/SENSOR01 restart	PASS
Suricata pipeline	Custom IDS alert reached Wazuh rule 86601	PASS
Zeek telemetry	conn.log/dns.log JSON generated and collected by Wazuh	PASS
Agent reconnect	FILE01 + SALES-PC01 returned Active after startup	PASS

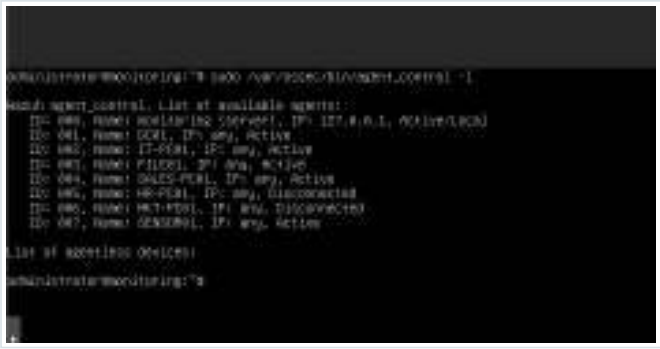


Figure 7A. FILE01 and SALES-PC01 automatically reconnected to Wazuh.

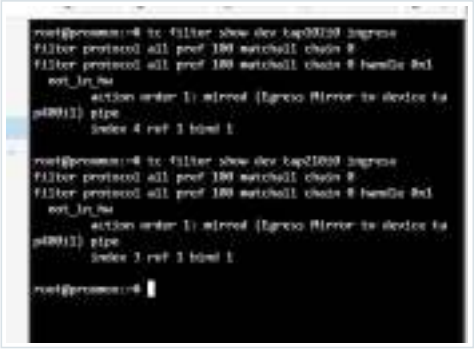


Figure 7B. Corresponding tc mirror rules appeared automatically on Proxmox.

ENGINEERING LESSONS

Major Troubleshooting & Root-Cause Analysis

Time synchronization prerequisite

Windows and Linux timestamps were normalized before trusting correlation. DC01 became the domain time authority and members synced through the domain hierarchy.

Malformed Wazuh XML

A closing-tag typo in FILE01 ossec.conf stopped WazuhSvc. Reading the agent error log and validating the XML corrected the issue without reinstalling the agent.

Incorrect agent manager address

MKT-PC01 was configured with 0.0.0.0 as the Wazuh manager. Correcting the address to 10.10.30.100 restored connectivity.

Raw telemetry vs. alert indexing

auditd ingestion worked even when a specific watched event was not immediately visible in the alerts index. Distinguishing collection, decoding, and rule matching prevented a false diagnosis.

Persistent packet mirroring

TAP interfaces exist only while VMs run. A boot-only rule would be fragile, so I used Proxmox hook scripts to rebuild tc/mirred rules on lifecycle events and verified idempotent behavior.

Suricata startup failure

The default service expected eth0 and had no downloaded rule file. Binding AF_PACKET to ens19 and running suricata-update resolved the failure.

Zeek downstream format

Native TSV logs were useful locally, but JSON logging made the data suitable for structured Wazuh collection and future detection/hunting projects.

What I Learned

- Visibility is an engineered system: time, endpoint policy, packet access, parsing, and transport all have to agree.
- A passive sensor should fail open from the network perspective; monitoring failure must not become connectivity failure.
- Telemetry collection and detection are separate concerns. SOC01 establishes the platform; focused DET projects will add meaningful analytics on top.

RECOVERY & HANDOFF

SOC01-COMPLETE-V1 Recovery Baseline

After final validation, I created a new recovery checkpoint before beginning detection-engineering projects. The backup set includes all nine VM backups plus host-side configuration that VM backups do not protect: Proxmox networking, qemu-server VM definitions, vmbr4, the persistent mirror hook, network state, and VM-to-hook mapping.

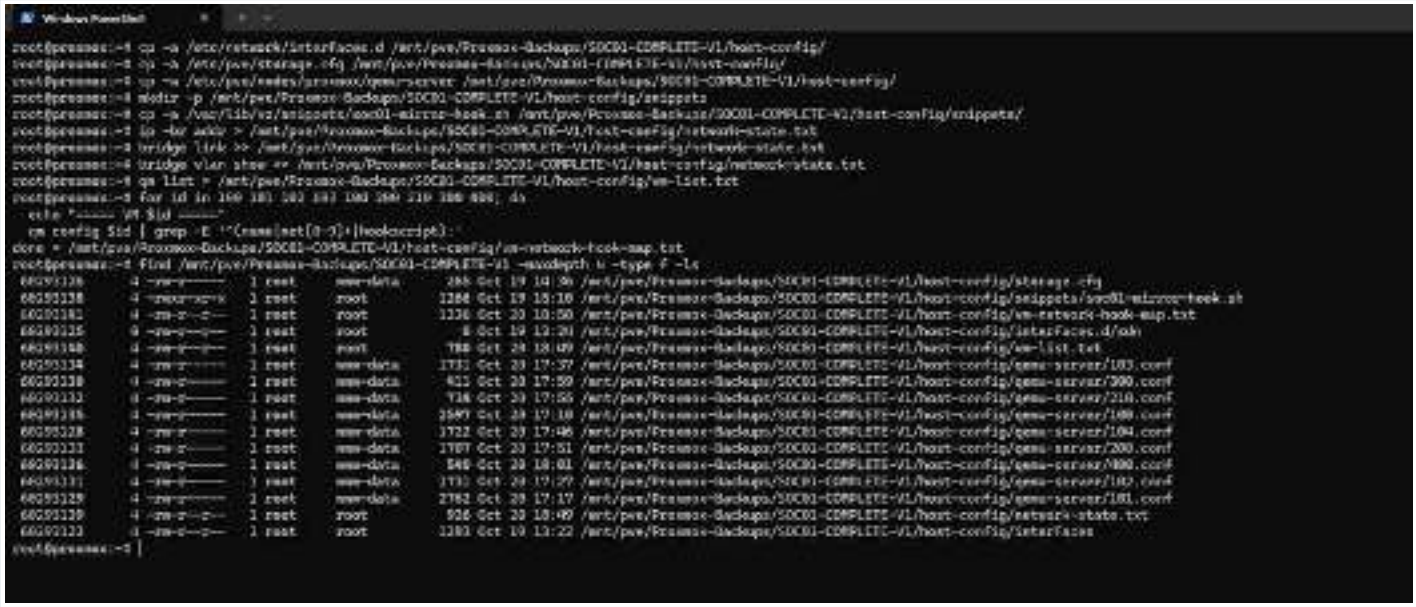


Figure 8. Final SOC01-COMPLETE-V1 host configuration and mirror-hook recovery package.

RECOVERY MODEL

SOC01-COMPLETE-V1 freezes the monitoring platform before DET01. The checkpoint preserves both guest systems and the Proxmox host configuration needed to reconstruct the passive sensor path.

Project Result

SOC01 is a validated enterprise-style security-monitoring platform with centralized Windows and Linux telemetry, passive packet visibility, Suricata IDS alerts, Zeek network metadata, and automated recovery of mirror rules when monitored systems return online.

NEXT PORTFOLIO MILESTONE

DET01 - Authentication Attack Detection & Investigation. SOC01 will remain the reusable infrastructure layer while detection engineering, tuning, ATT&CK mapping, hunting, and investigation are documented as focused projects.

Built, validated, troubleshot, and documented by Carlos Tiburcio